

Domain Driven Design

- Amjad Hossain

Let's **INVENT**(??) a *Problem Domain*

The GREAT and Unique

eCommerce Order Fulfillment System

A System that no one built before !!!

Why this particular *Problem Domain* ???

1. <https://fabiofumarola.github.io/nosql/readingMaterial/Evans03.pdf>
2. <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/ddd-oriented-microservice>
3. <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis>
4. <https://2017.dddeurope.com/speakers/udi-dahan/>
5. Mark the names: Eric Evans, Martin Fowler and Udi Dahan

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart
- Place the Order from online

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart
- Place the Order from online
- Make the payment

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart
- Place the Order from online
- Make the payment
- Check the status of the delivery

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart
- Place the Order from online
- Make the payment
- Check the status of the delivery
- Cancel the order

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart
- Place the Order from online
- Make the payment
- Check the status of the delivery
- Cancel the order
- Return the order

Let the **business requirements** arrive as usual

As a user I will be able to -

- Browse different type of product
- Select a single product and view details
- Add 1 or multiple product into the Cart
- Place the Order from online
- Make the payment
- Check the status of the delivery
- Cancel the order
- Return the order
- etc

Define The **Scoped Problem Domain**

We're building an **eCommerce Order Fulfilment System** with following capabilities:

- Customer places an order

Define The **Scoped Problem Domain**

We're building an **eCommerce Order Fulfilment System** with following capabilities:

- Customer places an order
- System reserves inventory

Define The **Scoped Problem Domain**

We're building an **eCommerce Order Fulfilment System** with following capabilities:

- Customer places an order
- System reserves inventory
- Customer completes payment

Define The **Scoped Problem Domain**

We're building an **eCommerce Order Fulfilment System** with following capabilities:

- Customer places an order
- System reserves inventory
- Customer completes payment
- Warehouse ships the order

Define The **Scoped Problem Domain**

We're building an **eCommerce Order Fulfilment System** with following capabilities:

- Customer places an order
- System reserves inventory
- Customer completes payment
- Warehouse ships the order
- Customer may cancel before shipment

Define The **Scoped Problem Domain**

We're building an **eCommerce Order Fulfilment System** with following capabilities:

- Customer places an order
- System reserves inventory
- Customer completes payment
- Warehouse ships the order
- Customer may cancel before shipment
- Customer may return after delivery

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.
- **SKU**: Stock Keeping Unit, the unique ID for a product used by inventory.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.
- **SKU**: Stock Keeping Unit, the unique ID for a product used by inventory.
- **InventoryItem**: stock state for a single SKU.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.
- **SKU**: Stock Keeping Unit, the unique ID for a product used by inventory.
- **InventoryItem**: stock state for a single SKU.
- **StockReservation**: a hold on stock for an order.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.
- **SKU**: Stock Keeping Unit, the unique ID for a product used by inventory.
- **InventoryItem**: stock state for a single SKU.
- **StockReservation**: a hold on stock for an order.
- **Shipment**: the fulfillment unit created for an order.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.
- **SKU**: Stock Keeping Unit, the unique ID for a product used by inventory.
- **InventoryItem**: stock state for a single SKU.
- **StockReservation**: a hold on stock for an order.
- **Shipment**: the fulfillment unit created for an order.
- **ShipmentItem**: one product entry inside the shipment.

Step 1: Understanding the Domain (**Ubiquitous Language**)

- **Order**: the customer purchase request and its lifecycle.
- **OrderLine**: one product entry inside the order.
- **PaymentAttempt**: one attempt to pay for the order.
- **SKU**: Stock Keeping Unit, the unique ID for a product used by inventory.
- **InventoryItem**: stock state for a single SKU.
- **StockReservation**: a hold on stock for an order.
- **Shipment**: the fulfillment unit created for an order.
- **ShipmentItem**: one product entry inside the shipment.
- **Stock**: available inventory

Step 2: Identifying **Subdomains** and **Bounded Context**

- **Ordering:** manages order lifecycle

Step 2: Identifying **Subdomains** and **Bounded Context**

- **Ordering:** manages order lifecycle
- **Inventory:** manages stock

Step 2: Identifying **Subdomains** and **Bounded Context**

- **Ordering**: manages order lifecycle
- **Inventory**: manages stock
- **Payment**: handles transactions

Step 2: Identifying **Subdomains** and **Bounded Context**

- **Ordering:** manages order lifecycle
- **Inventory:** manages stock
- **Payment:** handles transactions
- **Shipping:** handles delivery

Step 3: Define **Invariants**

- An order cannot be paid twice.
- Stock cannot be reserved beyond available quantity.
- A shipment cannot be marked shipped before it exists.
- Inventory must not reserve the item if payment failed
- An order should not move to a shipment-ready state until payment is confirmed and required inventory is reserved.
- Stock cannot go negative

Step 4: Identify **Aggregate Boundaries**

- **Order Aggregate:** Order(Entity), OrderItem, OrderState/OrderStatus

Step 4: Identify **Aggregate Boundaries**

- **Order Aggregate:** Order(Entity), OrderItem, OrderState/OrderStatus
- **Inventory Aggregate:** Inventory/Stock(Entity), InventoryItem, StockReservation

Step 4: Identify **Aggregate Boundaries**

- **Order Aggregate:** Order(Entity), OrderItem, OrderState/OrderStatus
- **Inventory Aggregate:** Inventory/Stock(Entity), InventoryItem, StockReservation
- **Shipment Aggregate:** Shipment(Entity), ShippingItem, ShippingState

Step 4: Identify **Aggregate Boundaries**

- **Order Aggregate:** Order(Entity), OrderItem, OrderState/OrderStatus
- **Inventory Aggregate:** Inventory/Stock(Entity), InventoryItem, StockReservation
- **Shipment Aggregate:** Shipment(Entity), ShippingItem, ShippingState
- **Payment Aggregate:** Payment(Entity), PaymentAttempt, PaymentMethod

Step 5: Define few **Aggregate Behaviours**

- **Order Aggregate:** place(), draft(), confirm()

Step 5: Define few **Aggregate Behaviours**

- **Order Aggregate:** place(), draft(), confirm()
- **Inventory Aggregate:** reserve(), releaseReservation()

Step 5: Define few **Aggregate Behaviours**

- **Order Aggregate:** place(), draft(), confirm()
- **Inventory Aggregate:** reserve(), releaseReservation()
- **Shipment Aggregate:** create(), markShipped(), return()

Step 5: Define few **Aggregate Behaviours**

- **Order Aggregate:** place(), draft(), confirm()
- **Inventory Aggregate:** reserve(), releaseReservation()
- **Shipment Aggregate:** create(), markShipped(), return()
- **Payment Aggregate:** initiate(), failed(), completed(), pending()

Step 6: Define **Repositories**

- **OrderRepository**
- **InventoryRepository**
- **PaymentRepository**
- **ShipmentRepository**

Step 7: Define few **Use Cases**

- **PlaceOrderUseCase**
- **ReserveInventoryUseCase**
- **ConfirmPaymentUseCase**
- **MarkShipmentShippedUseCase**

Put logic in the aggregate when:

- **It protects the aggregate's own invariants**
- **It changes aggregate's internal state**
- **It deals with the data already inside the aggregate**

Put logic in the UseCases when:

- **the workflow spans multiple aggregates**
- **you need multiple repositories**
- **you coordinate transaction boundaries**
- **you call external systems**
- **you orchestrate steps, but do not own deep business rules**

Don't mix up

Application Service / Use Case: Coordinates a business use case.

Domain Service: Used only when a real business rule exists, but it does **not naturally** belong to one aggregate/value object/entity.

Infrastructure Service: Technical integration

- payment gateway client
- email sender
- message broker adapter

Place Order flow

OrderController

→ PlaceOrderUseCase

→ **OrderRepository**.load(...)

→ **InventoryRepository**.load(...)

→ **OrderAggregate**.place(...)

→ **InventoryAggregate**.reserve(...)

→ **OrderRepository**.save(order)

→ **InventoryRepository**.save(inventory)

Confirm Payment flow

PaymentController

→ **ConfirmPaymentUseCase**

→ **PaymentRepository**.load(...)

→ **OrderRepository**.load(...)

→ **PaymentAggregate**.complete(...)

→ **OrderAggregate**.confirm(...)

→ **PaymentRepository**.save(payment)

→ **OrderRepository**.save(order)

Create Shipment flow

ShipmentController

→ **CreateShipmentUseCase**

→ **OrderRepository**.load(...)

→ **OrderAggregate**.ensureReadyForShipment(...)

→ **ShipmentAggregate**.create(...)

→ **ShipmentRepository**.save(shipment)

Cancel Order flow

OrderController

→ CancelOrderUseCase

→ **OrderRepository**.load(...)

→ **InventoryRepository**.load(...)

→ **OrderAggregate**.cancel(...)

→ **InventoryAggregate**.releaseReservation(...)

→ **OrderRepository**.save(order)

→ **InventoryRepository**.save(inventory)

```

src/
├── EcommerceDddDemo/
│   ├── Domain/
│   │   ├── Ordering/
│   │   │   ├── Order.cs
│   │   │   ├── OrderLine.cs
│   │   │   ├── OrderStatus.cs
│   │   │   ├── OrderId.cs
│   │   │   └── IOrderRepository.cs
│   │   ├── Inventory/
│   │   │   ├── Inventory.cs
│   │   │   ├── StockReservation.cs
│   │   │   ├── Sku.cs
│   │   │   ├── Quantity.cs
│   │   │   └── IInventoryRepository.cs
│   │   └── Shared/
│   │       └── Money.cs
│   ├── Application/
│   │   ├── Orders/
│   │   │   ├── PlaceOrder/
│   │   │   │   ├── PlaceOrderCommand.cs
│   │   │   │   └── PlaceOrderUseCase.cs
│   │   │   └── ConfirmPayment/
│   │   │       ├── ConfirmPaymentCommand.cs
│   │   │       └── ConfirmPaymentUseCase.cs
│   ├── Infrastructure/
│   │   ├── Repositories/
│   │   │   ├── InMemoryOrderRepository.cs
│   │   │   └── InMemoryInventoryRepository.cs
│   │   └── PaymentGateway/
│   │       └── FakePaymentGateway.cs

```

```

1 src/
2   └─ EcommerceBdd/
3       └─ Shared/
4
5       └─ Domain/
6           └─ Application/
7               └─ Infrastructure/
8
9       └─ Modules/
10
11         └─ Ordering/
12             └─ Domain/
13                 └─ Aggregates/
14                     └─ Order.cs
15                 └─ Entities/
16                     └─ OrderLine.cs
17                 └─ ValueObjects/
18                     └─ OrderId.cs
19                     └─ OrderStatus.cs
20                     └─ Money.cs
21                 └─ Repositories/
22                     └─ IOrderRepository.cs
23
24             └─ Application/
25                 └─ Commands/
26                     └─ PlaceOrderCommand.cs
27                 └─ UseCases/
28                     └─ PlaceOrderUseCase.cs
29
30             └─ Infrastructure/
31                 └─ Repositories/
32                     └─ InMemoryOrderRepository.cs
33
34             └─ Module.cs
35
36         └─ Inventory/
37             └─ Domain/
38                 └─ Aggregates/
39                     └─ Inventory.cs
40                 └─ Entities/
41                     └─ StockReservation.cs
42                 └─ ValueObjects/
43                     └─ sku.cs
44                     └─ Quantity.cs
45                 └─ Repositories/
46                     └─ IInventoryRepository.cs
47
48             └─ Application/
49                 └─ UseCases/
50                     └─ ReserveStockUseCase.cs
51
52             └─ Infrastructure/
53                 └─ Repositories/
54                     └─ InMemoryInventoryRepository.cs
55
56             └─ Module.cs
57
58         └─ Payment/
59             └─ Domain/
60                 └─ Aggregates/
61                     └─ Payment.cs
62                 └─ Entities/
63                     └─ PaymentAttempt.cs
64                 └─ ValueObjects/
65                     └─ PaymentId.cs

```